

TÀI LIỆU KỸ THUẬT NHẬP MÔN

HIỂU VÀ ỨNG DỤNG AI TRONG CÔNG VIỆC

Từ mô hình ngôn ngữ đến hệ thống AI có thể kiểm soát

Dành cho kỹ sư phần mềm và người xây dựng sản phẩm

Bản biên tập mở rộng - 2026

Lời mở đầu

AI hiện đại thường được giới thiệu bằng tên mô hình, bảng xếp hạng hoặc những màn trình diễn ấn tượng. Cách tiếp cận đó giúp người đọc biết AI có thể làm gì, nhưng chưa đủ để trả lời ba câu hỏi quan trọng hơn: hệ thống vận hành theo quy luật nào, mỗi thành phần chịu trách nhiệm gì, và làm thế nào để kiểm soát chất lượng khi đưa AI vào công việc thật.

Cuốn sách này tiếp cận AI như một **hệ thống kỹ thuật xác suất**. Mô hình ngôn ngữ là lõi tạo sinh; prompt chỉ dẫn nhiệm vụ; context cung cấp dữ kiện; retrieval tìm tri thức liên quan; workflow điều phối các bước; tool nối AI với thế giới bên ngoài; evaluation đo chất lượng; guardrail giới hạn rủi ro. Khi nhìn thấy đúng ranh giới trách nhiệm, người học có thể tự thiết kế giải pháp thay vì phụ thuộc vào một sản phẩm hay framework cụ thể.

Một ví dụ xuyên suốt được dùng trong sách là **trợ lý tra cứu tài liệu kỹ thuật nội bộ**. Hệ thống nhận câu hỏi của kỹ sư, tìm đoạn tài liệu liên quan, tạo câu trả lời có dẫn nguồn và chuyển các yêu cầu nguy hiểm cho con người phê duyệt. Ví dụ này đủ nhỏ để hiểu, nhưng chứa gần như đầy đủ các thành phần của một ứng dụng AI sản xuất.

Nguyên tắc đọc: Đừng hỏi “AI thông minh đến đâu?” trước khi hỏi “dữ liệu nào đi vào, thành phần nào quyết định, kết quả được kiểm tra bằng gì, và ai chịu trách nhiệm khi sai?”.

Cách sử dụng tài liệu

- Nếu mới bắt đầu, đọc tuần tự từ Chương 1 đến Chương 5.
- Nếu đang xây sản phẩm, tập trung Chương 5 đến Chương 10.
- Nếu cần chọn công nghệ, đọc Chương 13 và bảng framework ở Phụ lục C.
- Nếu gặp thuật ngữ lạ, tra Phụ lục A; mỗi mục có nghĩa, vai trò và khái niệm liên quan.

Mục lục

Lời mở đầu	i
Cách sử dụng tài liệu	ii
1 Nền tảng tư duy về AI	1
1.1 AI là gì?	1
1.1.1 Nhiệm vụ và vai trò	2
1.1.2 Ví dụ ngắn	2
1.2 AI khác phần mềm truyền thống ở đâu?	2
1.3 AI không phải là gì?	2
1.4 Các làn sóng phát triển chính	3
1.5 Các nhánh chính và nhiệm vụ	3
2 Generative AI và mô hình ngôn ngữ lớn	4
2.1 Generative AI là gì?	4
2.2 Large Language Model (LLM)	4
2.3 LLM sinh câu trả lời như thế nào?	5
2.4 Khả năng và giới hạn	5
2.5 Chọn mô hình theo bài toán	6
3 Token, context và cơ chế tạo sinh	7
3.1 Token và tokenizer	7
3.2 Context window	7
3.3 Transformer và attention	7
3.4 Predict next token	8
3.5 Temperature, top-p và tính lặp lại	8
3.6 Hallucination và các kiểu sai	8
3.7 Vì sao AI “quên”?	8
4 Giao tiếp hiệu quả với AI	10
4.1 Prompt là gì?	10
4.2 Một prompt có cấu trúc	10
4.3 Zero-shot, one-shot và few-shot	11
4.4 Structured output	11
4.5 Prompt Engineering	11
5 Context Engineering và RAG	12
5.1 Context Engineering	12
5.2 Grounding	12
5.3 RAG là gì?	12
5.3.1 Luồng lập chỉ mục	13

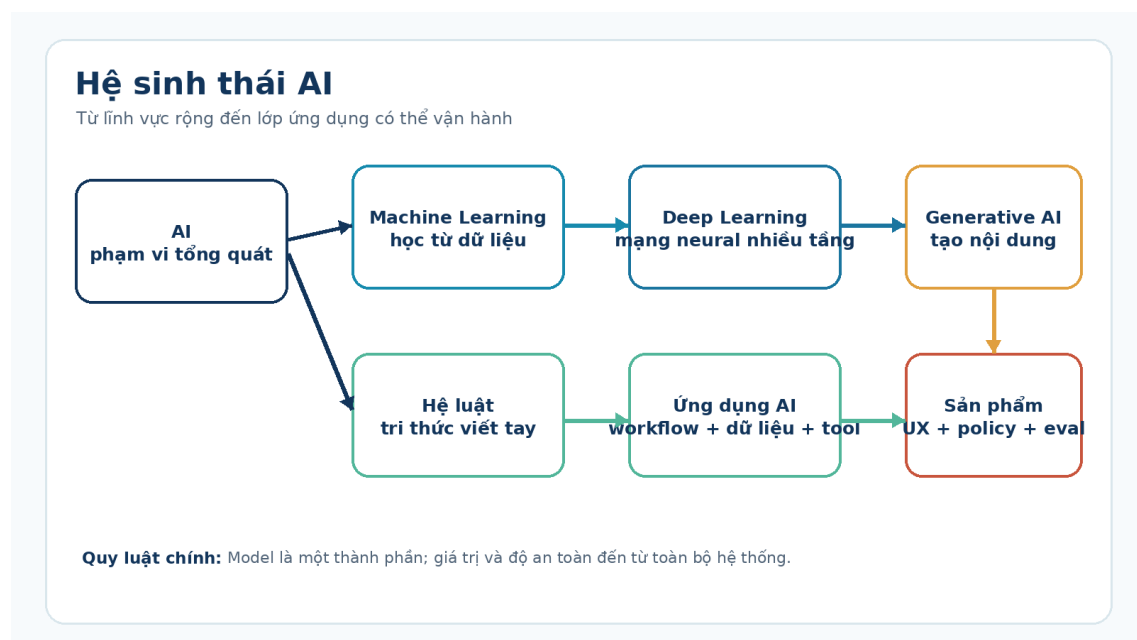
5.3.2	Luồng truy vấn	13
5.4	Embedding	13
5.5	Chunking, metadata và retrieval	14
5.6	Vector store và search engine	14
5.7	Đánh giá RAG	14
6	Workflow Engineering	15
6.1	Workflow là gì?	15
6.2	Thành phần cốt lõi	15
6.3	Khi nào nên chia nhỏ?	15
6.4	Workflow xác định và workflow có AI	16
7	Agent Engineering	17
7.1	Agent là gì?	17
7.2	Vòng lặp agent	17
7.3	Tool calling và function calling	18
7.4	MCP	18
7.5	Memory và state	18
7.6	Multi-agent	18
7.7	Khi nào không nên dùng agent?	19
8	Loop Engineering và kiểm soát sai số	20
8.1	Loop là gì?	20
8.2	Reflection, critic, validator và repair	20
8.3	Ví dụ vòng lặp sửa code	20
9	Thiết kế hệ thống AI	22
9.1	Kiến trúc tham chiếu	22
9.2	Model gateway	23
9.3	Tool gateway	23
9.4	Guardrail	23
9.5	Prompt injection và dữ liệu không tin cậy	23
9.6	Reliability, latency và cost	24
9.7	Observability	24
10	Evaluation: đo chất lượng thay vì đo cảm giác	25
10.1	Vì sao evaluation là thành phần trung tâm?	25
10.2	Xây bộ test	25
10.3	Các loại evaluator	26
10.4	Offline và online evaluation	26
10.5	Ngưỡng phát hành	26
11	Quy trình phát triển sản phẩm AI	27
11.1	Bước 1 - Xác định bài toán và ranh giới	27
11.2	Bước 2 - Xây baseline đơn giản	27
11.3	Bước 3 - Tạo evaluation set sớm	27
11.4	Bước 4 - Thiết kế prompt, context và tool	27
11.5	Bước 5 - Chọn workflow hoặc agent	27
11.6	Bước 6 - Kiểm thử nhiều lớp	27
11.7	Bước 7 - Triển khai có kiểm soát	28
11.8	Bước 8 - Học từ lỗi thực tế	28
12	Các mẫu ứng dụng AI trong thế giới thực	29

12.1 Trợ lý tri thức nội bộ	29
12.2 Intelligent Document Processing	29
12.3 AI coding assistant	29
12.4 Hỗ trợ chăm sóc khách hàng	29
12.5 Phân tích nghiên cứu	30
13 Lựa chọn framework và công cụ	31
13.1 LangChain	31
13.2 LlamaIndex	31
13.3 LangGraph	31
13.4 OpenAI Agents SDK	32
13.5 MCP	32
13.6 n8n	32
13.7 Flowise	33
13.8 Dify	33
13.9 Haystack	33
13.10 Ma trận chọn nhanh	33
A Bảng tra cứu thuật ngữ	35
B Phân biệt các khái niệm thường nhầm	40
C Bản đồ trách nhiệm trong hệ sinh thái	41
D Tài liệu tham khảo	42
D.1 Tài liệu framework và giao thức	42
D.2 Nền tảng học thuật và kỹ nghệ	42
Kết luận	43

CHƯƠNG 1

Nền tảng tư duy về AI

Mục tiêu của phần: phân biệt AI với phần mềm truyền thống, hiểu vị trí của Machine Learning, Deep Learning và Generative AI, đồng thời hình thành cách nhìn đúng về năng lực và giới hạn của hệ thống xác suất.



Hình 1.1: Bản đồ khái quát hệ sinh thái AI

1.1 AI là gì?

Trí tuệ nhân tạo (Artificial Intelligence - AI) là lĩnh vực xây dựng hệ thống có thể thực hiện những nhiệm vụ thường cần đến nhận thức của con người, chẳng hạn nhận diện hình ảnh, hiểu ngôn ngữ, dự đoán, lập kế hoạch hoặc đưa ra khuyến nghị.

AI là một phạm vi rộng. Bên trong nó có nhiều cách tiếp cận:

- **Hệ luật:** chuyên gia viết quy tắc rõ ràng để máy suy luận.
- **Machine Learning:** thuật toán học quan hệ từ dữ liệu thay vì nhận toàn bộ quy tắc viết tay.
- **Deep Learning:** Machine Learning dùng mạng neural nhiều tầng để học biểu diễn phức tạp.

- **Generative AI:** mô hình học cấu trúc của dữ liệu để tạo ra nội dung mới như văn bản, hình ảnh, âm thanh hoặc mã nguồn.

1.1.1 Nhiệm vụ và vai trò

Trong kiến trúc phần mềm, AI phù hợp với phần việc có đầu vào nhiều biến thể, quy tắc khó mô tả đầy đủ hoặc cần xử lý dữ liệu phi cấu trúc. AI **không thay thế toàn bộ logic nghiệp vụ**. Tính thuế, kiểm tra quyền truy cập hay đối soát sổ dư vẫn nên dùng quy tắc xác định; AI có thể hỗ trợ đọc chứng từ, phân loại yêu cầu hoặc phát hiện bất thường.

1.1.2 Ví dụ ngắn

Bài toán	Thành phần phù hợp	Lý do
Tính tổng tiền hóa đơn	Mã nguồn xác định	Công thức rõ và phải chính xác
Trích trường dữ liệu từ nhiều mẫu hóa đơn	AI + validator	Bố cục biến đổi, cần hiểu ngữ cảnh
Quyết định thanh toán	Luật nghiệp vụ + con người	Rủi ro tài chính và pháp lý cao

1.2 AI khác phần mềm truyền thống ở đâu?

Phần mềm truyền thống thường ánh xạ đầu vào sang đầu ra bằng quy tắc do lập trình viên xác định. Với cùng dữ liệu và cùng trạng thái, kết quả thường lặp lại. Mô hình AI học các tham số từ dữ liệu và tạo dự đoán theo phân phối xác suất. Vì vậy, chất lượng được mô tả bằng tỷ lệ đúng, độ bao phủ, sai số và mức tin cậy - không chỉ bằng “chạy” hoặc “không chạy”.

Sự khác biệt này dẫn đến ba hệ quả:

1. **Cần dữ liệu đánh giá đại diện.** Một vài ví dụ đẹp không chứng minh hệ thống hoạt động tốt ngoài thực tế.
2. **Cần kiểm soát biên lỗi.** Đầu ra phải được xác thực bằng schema, luật nghiệp vụ hoặc con người tùy mức rủi ro.
3. **Cần giám sát sau triển khai.** Dữ liệu người dùng và hành vi mô hình có thể thay đổi theo thời gian.

1.3 AI không phải là gì?

AI tạo sinh hiện nay không nên được mặc định là có ý thức, hiểu thế giới như con người hoặc luôn biết sự thật. LLM tạo chuỗi token phù hợp với ngữ cảnh và quá trình huấn luyện. Khả năng ngôn ngữ mượt mà có thể tạo cảm giác hiểu sâu, nhưng câu trả lời vẫn có thể sai dữ kiện, bỏ sót điều kiện hoặc suy luận không nhất quán.

Một số cách hiểu cần tránh:

- **“AI chỉ là một chuỗi if-else lớn.”** Không đúng: mô hình học tham số số học, không lưu toàn bộ hành vi dưới dạng luật đọc được.

- **“AI hiểu nghĩa chỉ bằng khoảng cách vector.”** Quá đơn giản: embedding là một lớp biểu diễn; hành vi còn đến từ attention, tham số đã học, ngữ cảnh và quá trình giải mã.
- **“Càng nhiều dữ liệu càng tốt.”** Chưa đủ: dữ liệu phải phù hợp, có nguồn gốc rõ, đại diện cho bài toán và được kiểm soát sai lệch.
- **“RAG loại bỏ hallucination.”** Không đúng: RAG giảm rủi ro thiếu dữ kiện nhưng có thể truy xuất sai, lấy thiếu tài liệu hoặc tổng hợp sai.

1.4 Các làn sóng phát triển chính

1. **Hệ chuyên gia và AI dựa trên luật:** tri thức được mã hóa thủ công; dễ giải thích nhưng khó mở rộng khi số luật tăng.
2. **Machine Learning thống kê:** hệ thống học mẫu từ dữ liệu có cấu trúc; nổi bật trong phân loại, dự báo và xếp hạng.
3. **Deep Learning:** học biểu diễn từ dữ liệu lớn; thúc đẩy thị giác máy tính, nhận dạng tiếng nói và NLP.
4. **Mô hình nền tảng và Generative AI:** một mô hình lớn có thể thích nghi với nhiều tác vụ qua prompt, context, tool hoặc fine-tuning.

Lịch sử này cho thấy AI không phát triển bằng cách loại bỏ hoàn toàn phương pháp cũ. Hệ thống sản xuất thường kết hợp luật, mô hình học máy, LLM, cơ sở dữ liệu và quy trình phê duyệt.

1.5 Các nhánh chính và nhiệm vụ

Nhánh	Nhiệm vụ điển hình	Vai trò trong hệ thống
Machine Learning	dự báo, phân loại, xếp hạng	Học quan hệ từ dữ liệu có cấu trúc hoặc đặc trưng
Deep Learning	nhận dạng ảnh, tiếng nói, ngôn ngữ	Học biểu diễn phức tạp từ dữ liệu lớn
NLP	phân tích và sinh ngôn ngữ	Xử lý văn bản, lời nói, thực thể và ý định
Computer Vision	nhận diện, phân vùng, OCR	Chuyển ảnh/video thành thông tin có thể xử lý
Robotics	cảm nhận, lập kế hoạch, điều khiển	Kết nối quyết định số với hành động vật lý
Generative AI	sinh văn bản, ảnh, âm thanh, code	Tạo hoặc chuyển đổi nội dung theo điều kiện đầu vào

CHƯƠNG 2

Generative AI và mô hình ngôn ngữ lớn

2.1 Generative AI là gì?

Generative AI là nhóm mô hình học cấu trúc phân phối của dữ liệu để tạo mẫu mới có đặc điểm tương tự dữ liệu đã học. “Mới” không có nghĩa là hoàn toàn độc lập với dữ liệu huấn luyện; nó có nghĩa kết quả được tổng hợp từ các quy luật đã học thay vì lấy nguyên một bản ghi từ cơ sở dữ liệu.

Các họ mô hình thường gặp gồm mô hình tự hồi quy cho văn bản, diffusion cho hình ảnh và mô hình đa phương thức có thể xử lý nhiều loại dữ liệu. Trong ứng dụng, Generative AI thường thực hiện ba kiểu công việc:

- **Tạo:** viết bản nháp, sinh ảnh, tạo mã nguồn.
- **Biến đổi:** tóm tắt, dịch, đổi văn bản thành JSON, chuyển phong cách.
- **Tương tác:** trả lời câu hỏi, dùng công cụ, điều phối nhiều bước.

2.2 Large Language Model (LLM)

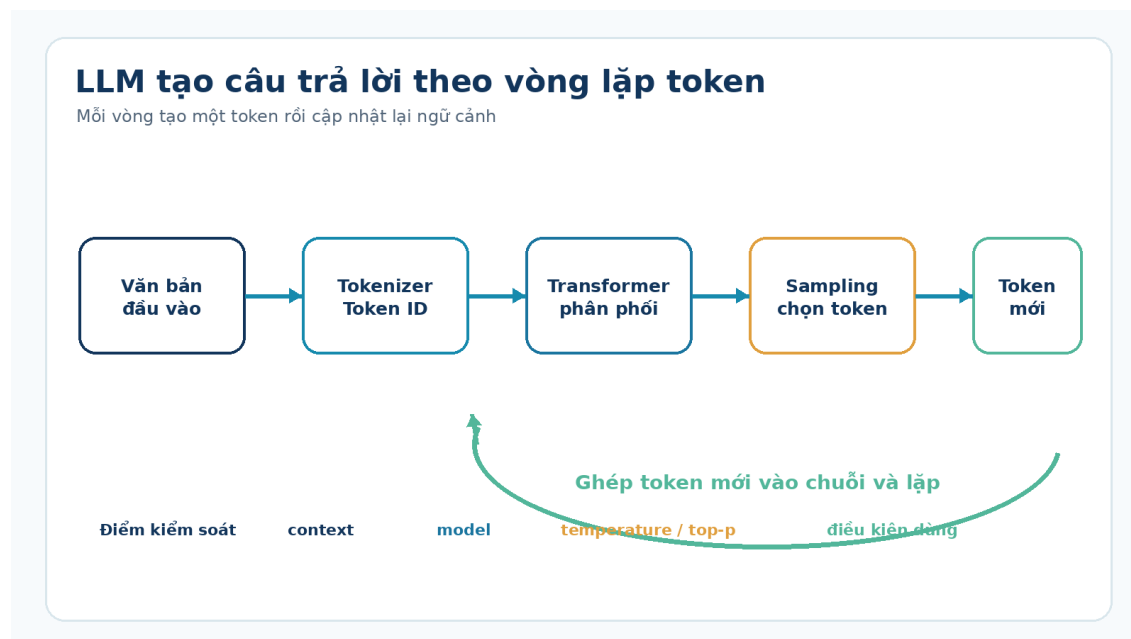
LLM là mô hình ngôn ngữ có số lượng tham số và dữ liệu huấn luyện đủ lớn để thực hiện nhiều tác vụ ngôn ngữ thông qua chỉ dẫn. Phần lớn LLM hiện đại dựa trên Transformer và được tiền huấn luyện bằng mục tiêu dự đoán token hoặc biến thể liên quan.

Ba lớp cần phân biệt:

Lớp	Ví dụ	Trách nhiệm
Mô hình	một LLM cụ thể	Tính toán phân phối token và biểu diễn
API/runtime	dịch vụ gọi mô hình	Xác thực, giới hạn, cấu hình sinh, tool calling
Ứng dụng	chatbot, trợ lý code, RAG	Quản lý người dùng, dữ liệu, workflow, kiểm soát rủi ro

Khi một chatbot trả lời sai, nguyên nhân có thể nằm ở bất kỳ lớp nào: mô hình yếu, context thiếu, retrieval sai, tool lỗi, prompt mơ hồ hoặc dữ liệu nguồn đã cũ. Không nên quy mọi lỗi về “mô hình không thông minh”.

2.3 LLM sinh câu trả lời như thế nào?



Hình 2.1: Chu trình sinh token của một LLM tự hồi quy

Quy trình khái quát:

1. **Tokenizer** biến văn bản thành các token ID.
2. Token được ánh xạ thành vector và xử lý qua các lớp Transformer.
3. Mô hình tạo phân phối xác suất cho token tiếp theo.
4. Bộ giải mã chọn một token theo chiến lược cấu hình.
5. Token mới được ghép vào chuỗi; vòng lặp tiếp tục đến điều kiện dừng.

Mô tả “dự đoán token tiếp theo” là đúng ở mức cơ chế, nhưng không có nghĩa LLM chỉ hoàn thành câu đơn giản. Trong quá trình huấn luyện, mô hình học nhiều mẫu về ngôn ngữ, mã nguồn, cấu trúc dữ liệu và quan hệ khái niệm. Năng lực phức tạp xuất hiện từ việc áp dụng lặp lại cơ chế dự đoán trong ngữ cảnh lớn.

2.4 Khả năng và giới hạn

LLM thường phù hợp với:

- tóm tắt, phân loại và trích xuất từ văn bản;
- chuyển đổi giữa các định dạng có cấu trúc;
- tạo bản nháp, gợi ý phương án và hỗ trợ lập trình;
- giao diện ngôn ngữ cho công cụ hoặc dữ liệu nội bộ.

LLM không nên tự chịu trách nhiệm cuối cùng khi:

- phép tính hoặc giao dịch phải chính xác tuyệt đối;
- quyết định ảnh hưởng trực tiếp đến an toàn, pháp lý hoặc tài chính;
- dữ liệu nguồn chưa được xác minh;
- độ trễ, chi phí hoặc quyền riêng tư không phù hợp.

Giải pháp thực tế không phải “dùng” hoặc “không dùng” LLM, mà là đặt LLM đúng vị trí. Hãy để mô hình xử lý phần mơ hồ; để code, cơ sở dữ liệu và con người xử lý phần

cần tính xác định và trách nhiệm.

2.5 Chọn mô hình theo bài toán

Không có mô hình tốt nhất cho mọi trường hợp. Cần đánh giá ít nhất sáu chiều:

1. **Chất lượng trên dữ liệu thật:** đo bằng bộ test của chính sản phẩm.
2. **Độ trễ:** thời gian đến token đầu tiên và tốc độ tạo kết quả.
3. **Chi phí:** input, output, cache, tool và hạ tầng liên quan.
4. **Context window:** dung lượng tối đa không đồng nghĩa với khả năng sử dụng mọi chi tiết như nhau.
5. **Khả năng điều khiển:** structured output, tool calling, multimodal, streaming.
6. **Ràng buộc vận hành:** riêng tư, khu vực dữ liệu, giấy phép, tự lưu trữ và hỗ trợ.

Một quy trình chọn tốt bắt đầu bằng 30-100 ca kiểm thử đại diện, sau đó mới so sánh mô hình. Benchmark công khai hữu ích để sàng lọc, nhưng không thay thế evaluation theo miền dữ liệu của sản phẩm.

CHƯƠNG 3

Token, context và cơ chế tạo sinh

3.1 Token và tokenizer

Token là đơn vị rời rạc mà mô hình dùng để biểu diễn chuỗi đầu vào và đầu ra. Một token có thể là cả từ, một phần của từ, dấu câu hoặc một chuỗi byte. Vì tokenizer khác nhau giữa các mô hình, không nên dùng quy đổi cố định như “một token luôn bằng 0,75 từ” cho mọi ngôn ngữ.

Vai trò: token là đơn vị tính dung lượng context, chi phí và giới hạn output. Văn bản tiếng Việt có dấu, mã nguồn hoặc chuỗi JSON có thể có tỷ lệ token khác tiếng Anh thông thường.

Ví dụ: cùng một yêu cầu dài 2.000 ký tự có thể tiêu tốn số token khác nhau ở hai tokenizer. Khi tối ưu chi phí, cần đo bằng tokenizer của mô hình đang dùng.

3.2 Context window

Context window là tổng số token mô hình có thể xử lý trong một lượt, thường gồm instruction, lịch sử hội thoại, tài liệu, kết quả tool và phần output dự kiến. Đây là **bộ đệm làm việc**, không phải bộ nhớ dài hạn tự động.

Context lớn giúp nạp nhiều dữ liệu hơn nhưng kéo theo chi phí, độ trễ và nguy cơ “nhiều ngữ cảnh”. Thông tin quan trọng có thể bị chìm giữa nhiều đoạn ít liên quan. Vì vậy, context engineering tập trung vào chọn đúng thông tin, không chỉ nạp nhiều thông tin.

3.3 Transformer và attention

Transformer là kiến trúc mạng neural xử lý chuỗi bằng attention và các lớp biến đổi học được. **Self-attention** cho phép mỗi vị trí trong chuỗi tính mức liên quan với các vị trí khác để tạo biểu diễn phụ thuộc ngữ cảnh.

Attention không phải bằng chứng rằng mô hình “chú ý” theo nghĩa tâm lý. Nó là phép tính trọng số giữa các biểu diễn. Vai trò của nó là giúp mô hình kết hợp thông tin ở nhiều vị trí và xử lý song song hiệu quả trong huấn luyện.

3.4 Predict next token

Với chuỗi token $w_1, w_2, \dots, w_n$, mô hình ước lượng:

$$P(w_{n+1} \mid w_1, w_2, \dots, w_n)$$

Sau khi chọn w_{n+1} , mô hình lặp lại phép tính cho token kế tiếp. Đầu ra dài vì vòng lặp diễn ra nhiều lần, không phải vì mô hình viết cả đoạn trong một phép tính duy nhất.

3.5 Temperature, top-p và tính lặp lại

- **Temperature** điều chỉnh độ sắc của phân phối trước khi lấy mẫu. Giá trị thấp thường làm đầu ra ổn định hơn; giá trị cao tăng đa dạng nhưng cũng tăng rủi ro lệch hướng.
- **Top-p** chỉ cho phép lấy mẫu từ nhóm token có tổng xác suất tích lũy đạt ngưỡng p .
- **Seed**, cấu hình dịch vụ và thay đổi hạ tầng cũng có thể ảnh hưởng tính lặp lại. Temperature bằng 0 không nên được xem là bảo đảm tuyệt đối rằng mọi lần gọi luôn giống nhau.

Với trích xuất JSON hoặc phân loại, thường dùng độ ngẫu nhiên thấp và schema chặt. Với brainstorming hoặc sáng tác, có thể tăng độ đa dạng nhưng vẫn cần kiểm tra đầu ra.

3.6 Hallucination và các kiểu sai

Hallucination là đầu ra có vẻ hợp lý nhưng không được dữ kiện hỗ trợ hoặc mâu thuẫn với thực tế. Thuật ngữ này không bao phủ mọi lỗi. Một hệ thống AI còn có thể:

- truy xuất nhầm tài liệu;
- bỏ sót điều kiện trong prompt;
- tính toán sai;
- dùng tool đúng tên nhưng sai tham số;
- tạo JSON hợp lệ cú pháp nhưng sai nghiệp vụ;
- trích nguồn đúng nhưng diễn giải quá mức.

Nguyên nhân thường là tổ hợp giữa dữ liệu huấn luyện, giới hạn mô hình, context thiếu hoặc nhiễu, chiến lược giải mã và thiết kế ứng dụng. Vì vậy, biện pháp kiểm soát cũng phải nhiều lớp: grounding, retrieval, validator, tool, evaluation và phê duyệt của con người.

3.7 Vì sao AI “quên”?

Từ “quên” thường chỉ ba hiện tượng khác nhau:

1. Nội dung đã nằm ngoài context window.
2. Nội dung vẫn còn nhưng không được mô hình ưu tiên đúng mức.
3. Ứng dụng không lưu hoặc không nạp lại trạng thái từ phiên trước.

Giải pháp tương ứng là rút gọn lịch sử, đánh dấu thông tin quan trọng, truy xuất bộ nhớ dài hạn hoặc thiết kế state store. Không nên cố giải quyết mọi trường hợp chỉ bằng cách tăng context window.

CHƯƠNG 4

Giao tiếp hiệu quả với AI

4.1 Prompt là gì?

Prompt là phần đầu vào có chủ đích dùng để hướng mô hình thực hiện một nhiệm vụ. Trong ứng dụng thực, “prompt” có thể gồm system instruction, developer instruction, tin nhắn người dùng, ví dụ, dữ liệu và mô tả công cụ. Vì vậy prompt không chỉ là một câu hỏi người dùng nhìn thấy.

4.2 Một prompt có cấu trúc

Thành phần	Câu hỏi cần trả lời	Ví dụ
Mục tiêu	Cần tạo ra kết quả gì?	Trích các lỗ hổng SQL injection
Bối cảnh	Hệ thống và dữ liệu nào liên quan?	PostgreSQL 15, đoạn code đính kèm
Tiêu chí	Thế nào là kết quả đạt?	Có vị trí, mức độ, giải thích ngắn
Ràng buộc	Không được làm gì?	Không sửa code, không suy đoán file khác
Định dạng	Hệ thống sau sẽ đọc gì?	JSON theo schema
Ví dụ	Mẫu nào thể hiện đúng ý?	Một input-output chuẩn

Mục tiêu: Rà soát đoạn code và liệt kê nguy cơ SQL injection.

Bối cảnh: Ứng dụng Node.js dùng PostgreSQL 15.

Tiêu chí: Mỗi phát hiện phải có dòng code, mức độ và lý do.

Ràng buộc: Chỉ dựa trên đoạn code được cung cấp; nếu thiếu dữ liệu, ghi rõ.

Đầu ra: JSON theo schema {findings: [{line, severity, reason}]}.

Input: <code>

Prompt tốt giảm mơ hồ, nhưng không thể bù cho dữ liệu sai hoặc thiếu năng lực. Khi yêu cầu phụ thuộc tài liệu nội bộ, cần context hoặc retrieval. Khi yêu cầu cần hành động, cần tool. Khi yêu cầu gồm nhiều bước và nhánh, cần workflow hoặc agent.

4.3 Zero-shot, one-shot và few-shot

- **Zero-shot:** chỉ cung cấp chỉ dẫn; phù hợp với tác vụ quen thuộc và định dạng đơn giản.
- **One-shot:** cung cấp một mẫu; hữu ích khi cần minh họa phong cách hoặc schema.
- **Few-shot:** cung cấp vài mẫu đa dạng; giúp mô hình nhận ra quy tắc khó diễn đạt bằng lời.

Ví dụ phải đại diện cho các trường hợp thật. Nhiều ví dụ gần giống nhau có thể làm mô hình bắt chước bề mặt nhưng không xử lý tốt trường hợp biên.

4.4 Structured output

Structured output buộc hoặc hướng mô hình trả kết quả theo schema có thể kiểm tra. Đây là cầu nối giữa ngôn ngữ xác suất và mã nguồn xác định.

Quy trình an toàn:

1. định nghĩa schema nhỏ, rõ kiểu dữ liệu;
2. yêu cầu mô hình trả theo schema;
3. parse và validate ở phía ứng dụng;
4. nếu lỗi, retry có giới hạn hoặc chuyển cho con người;
5. kiểm tra thêm quy tắc nghiệp vụ sau khi hợp lệ cú pháp.

JSON hợp lệ không đồng nghĩa nội dung đúng. Ví dụ total: -500 có thể đúng kiểu số nhưng không hợp lệ với nghiệp vụ hóa đơn.

4.5 Prompt Engineering

Prompt Engineering là hoạt động thiết kế và kiểm thử chỉ dẫn để mô hình thực hiện tác vụ ổn định hơn mà không thay đổi trọng số. Nó bao gồm cấu trúc yêu cầu, ví dụ, schema, cách xử lý thiếu dữ liệu và chiến lược kiểm thử.

Prompt nên được quản lý như mã nguồn: có phiên bản, review, test case và lịch sử thay đổi. Tránh “tối ưu bằng cảm giác” trên một vài hội thoại thuận lợi.

CHƯƠNG 5

Context Engineering và RAG

5.1 Context Engineering

Context Engineering là quá trình chọn, tổ chức và cung cấp đúng thông tin cùng đúng công cụ cho mô hình tại từng bước. Nếu Prompt Engineering trả lời “nên chỉ dẫn mô hình thế nào”, Context Engineering trả lời “mô hình cần thấy gì ngay lúc này để hoàn thành nhiệm vụ”.

Context có thể gồm:

- **Instruction:** mục tiêu, quy tắc và định dạng.
- **User context:** quyền hạn, ngôn ngữ, ưu tiên và trạng thái liên quan.
- **Task state:** bước hiện tại, kết quả trung gian, điều kiện đã thỏa.
- **Knowledge:** tài liệu, bản ghi hoặc dữ kiện được truy xuất.
- **Tool context:** công cụ khả dụng, schema tham số và kết quả gọi.
- **History:** phần hội thoại thật sự cần cho lượt hiện tại.

Vai trò của context builder là chọn phần cần thiết, loại trùng, giữ nguồn, sắp xếp ưu tiên và kiểm soát tổng số token. Nạp toàn bộ dữ liệu vào prompt thường vừa tốn chi phí vừa làm giảm tín hiệu.

5.2 Grounding

Grounding là cách neo câu trả lời vào nguồn dữ liệu được chỉ định, chẳng hạn tài liệu nội bộ, kết quả API hoặc hồ sơ người dùng. Một yêu cầu grounding tốt không chỉ nói “hãy dựa vào tài liệu”, mà còn quy định:

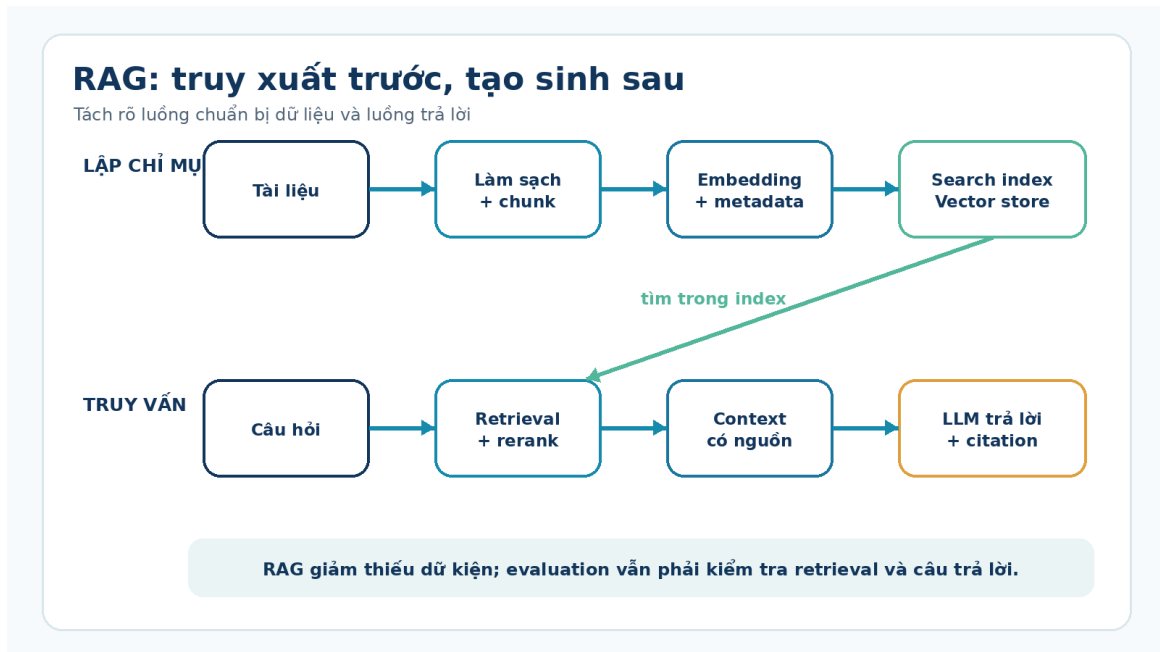
- nguồn nào được phép dùng;
- cách trích dẫn;
- phải làm gì khi nguồn thiếu hoặc mâu thuẫn;
- phần nào là dữ kiện, phần nào là suy luận.

Grounding giảm khả năng bịa dữ kiện nhưng không bảo đảm đúng tuyệt đối. Mô hình vẫn có thể hiểu sai đoạn nguồn hoặc ghép hai dữ kiện không tương thích.

5.3 RAG là gì?

Retrieval-Augmented Generation (RAG) là kiến trúc truy xuất dữ liệu liên quan tại thời điểm chạy và đưa dữ liệu đó vào context trước khi mô hình tạo câu trả lời.

RAG giúp cập nhật tri thức mà không cần huấn luyện lại mô hình và cho phép dẫn về nguồn gốc dữ liệu.



Hình 5.1: Pipeline RAG từ tài liệu đến câu trả lời có nguồn

RAG thường có hai luồng:

5.3.1 Luồng lập chỉ mục

1. thu nhận tài liệu và metadata;
2. làm sạch, chuẩn hóa;
3. chia tài liệu thành chunk;
4. tạo embedding hoặc chỉ mục từ khóa;
5. lưu vào vector store/search engine.

5.3.2 Luồng truy vấn

1. nhận câu hỏi và xác định phạm vi quyền truy cập;
2. tạo truy vấn tìm kiếm;
3. lấy ứng viên bằng vector, từ khóa hoặc hybrid search;
4. rerank, lọc trùng và chọn đoạn phù hợp;
5. xây context có nguồn;
6. sinh và kiểm tra câu trả lời.

5.4 Embedding

Embedding là vector số biểu diễn đặc điểm của một đối tượng như câu, đoạn văn, ảnh hoặc sản phẩm. Trong semantic search, các nội dung gần nghĩa thường có vector gần nhau theo một thước đo như cosine similarity.

Vai trò: embedding giúp tìm nội dung liên quan khi từ ngữ không trùng khớp hoàn toàn. Tuy nhiên, nó không phải một “tọa độ tri thức” tuyệt đối. Chất lượng phụ thuộc mô hình embedding, ngôn ngữ, miền dữ liệu và cách chia chunk.

Ví dụ: câu “không đăng nhập được sau khi đổi mật khẩu” có thể tìm thấy tài liệu “xử lý phiên đăng nhập hết hạn” dù không trùng nhiều từ khóa.

5.5 Chunking, metadata và retrieval

- **Chunking:** chia tài liệu thành đoạn đủ nhỏ để tìm chính xác nhưng đủ lớn để giữ ý nghĩa.
- **Metadata:** thông tin như tiêu đề, phiên bản, phòng ban, ngày hiệu lực và quyền truy cập.
- **Retrieval:** lấy các đoạn có khả năng liên quan nhất.
- **Reranking:** dùng một bộ chấm điểm tốt hơn để sắp xếp lại ứng viên.
- **Hybrid search:** kết hợp tìm từ khóa và vector để tận dụng ưu điểm của cả hai.

Không có kích thước chunk tốt cho mọi tài liệu. Hướng dẫn thao tác, API reference và hợp đồng pháp lý cần chiến lược khác nhau. Cần đánh giá retrieval độc lập trước khi đánh giá câu trả lời cuối.

5.6 Vector store và search engine

Vector store lưu vector cùng metadata và hỗ trợ tìm láng giềng gần. Một số hệ quản trị cơ sở dữ liệu hoặc search engine truyền thống cũng có khả năng vector search; vì vậy không phải dự án nào cũng cần một cơ sở dữ liệu vector chuyên dụng.

Tiêu chí chọn gồm quy mô dữ liệu, độ trễ, lọc metadata, hybrid search, quyền truy cập, sao lưu và năng lực vận hành của đội ngũ. Với dữ liệu nhỏ, giải pháp quen thuộc như PostgreSQL có extension vector có thể đơn giản hơn việc thêm một hệ thống mới.

5.7 Đánh giá RAG

Cần tách ít nhất ba lớp:

Lớp	Câu hỏi	Chỉ số gợi ý
Retrieval	Có lấy đúng đoạn nguồn không?	recall@k, precision@k, MRR
Generation	Câu trả lời có bám nguồn không?	faithfulness, correctness, citation accuracy
End-to-end	Người dùng giải quyết được việc không?	task success, thời gian, tỷ lệ chuyển người

Nếu retrieval không lấy được tài liệu đúng, thay prompt thường không giải quyết gốc vấn đề. Nếu retrieval đúng nhưng câu trả lời sai, cần xem prompt, model, context builder hoặc validator.

CHƯƠNG 6

Workflow Engineering

6.1 Workflow là gì?

Workflow là luồng xử lý gồm các bước, nhánh và điều kiện được thiết kế trước để biến đầu vào thành đầu ra. Một bước có thể là code thuần, truy vấn dữ liệu, gọi LLM, gọi API hoặc chờ con người phê duyệt.

Workflow hữu ích vì LLM thường ổn định hơn khi mỗi bước chỉ có một nhiệm vụ rõ. Hệ thống cũng dễ quan sát, retry và kiểm tra hơn so với một prompt khổng lồ làm mọi việc.

6.2 Thành phần cốt lõi

- **Trigger:** sự kiện bắt đầu, ví dụ email mới hoặc người dùng gửi yêu cầu.
- **Step/Node:** đơn vị xử lý có đầu vào và đầu ra rõ.
- **Router:** chọn nhánh theo điều kiện.
- **State:** dữ liệu cần giữ xuyên suốt luồng.
- **Validator:** kiểm tra schema và quy tắc nghiệp vụ.
- **Retry/Fallback:** xử lý lỗi có giới hạn.
- **Human-in-the-loop:** điểm dừng để con người xem hoặc phê duyệt.
- **Observability:** log, trace, chi phí, thời gian và kết quả từng bước.

6.3 Khi nào nên chia nhỏ?

Hãy chia khi tác vụ có các giai đoạn dùng tiêu chí khác nhau, ví dụ: trích dữ liệu, kiểm tra dữ liệu, đối chiếu cơ sở dữ liệu và viết giải thích. Không nên chia quá nhỏ nếu mỗi bước chỉ chuyển tiếp văn bản mà không tăng khả năng kiểm soát.

Ví dụ xử lý hóa đơn:

1. OCR lấy văn bản.
2. LLM trích các trường theo schema.
3. Code kiểm tra tổng tiền và mã số thuế.
4. Cơ sở dữ liệu đối chiếu nhà cung cấp.
5. Con người duyệt nếu số tiền vượt ngưỡng.

Mỗi thành phần làm đúng việc mình mạnh: AI xử lý bố cục và ngôn ngữ; code xử lý phép tính; database xác minh bản ghi; con người chịu trách nhiệm ở điểm rủi ro.

6.4 Workflow xác định và workflow có AI

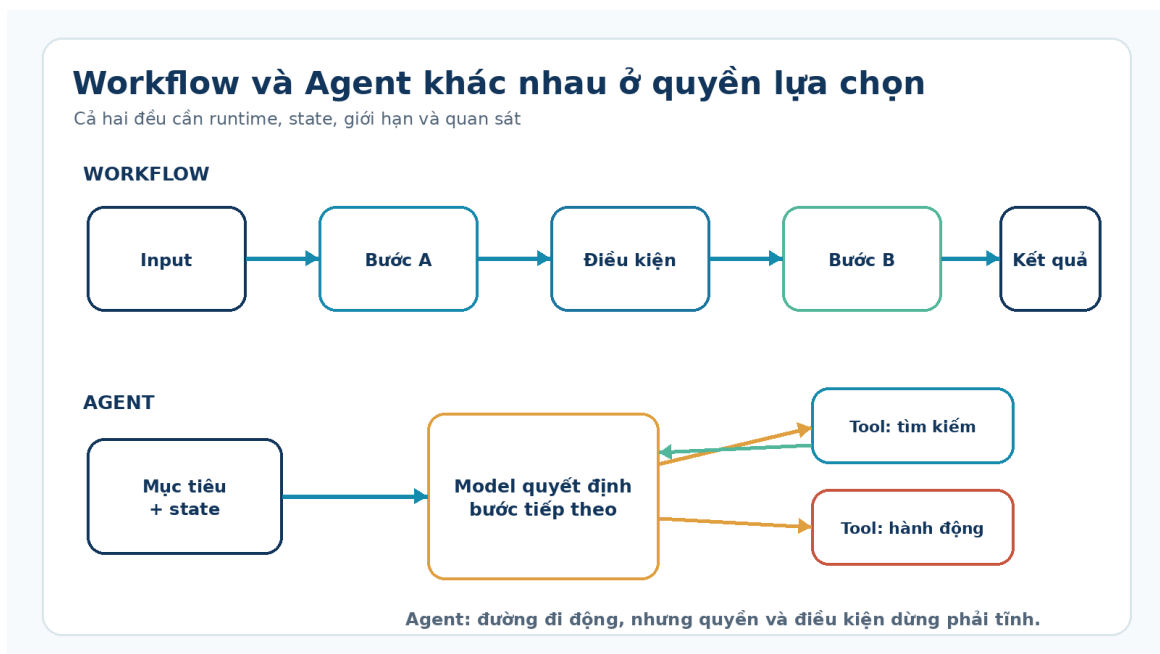
Workflow có AI vẫn cần phần khung xác định. Ví dụ, mô hình có thể phân loại email, nhưng router và quyền gửi phản hồi phải do ứng dụng kiểm soát. “AI workflow” không có nghĩa mọi nút đều là AI.

CHƯƠNG 7

Agent Engineering

7.1 Agent là gì?

AI agent là hệ thống cho phép mô hình lựa chọn hành động trong một vòng lặp để đạt mục tiêu. Hành động có thể là gọi tool, truy xuất dữ liệu, giao nhiệm vụ hoặc kết thúc. Mức tự chủ đến từ **quyền lựa chọn bước tiếp theo**, không chỉ từ việc dùng LLM.



Hình 7.1: Workflow xác định và agent lựa chọn công cụ

Một chatbot đơn giản nhận câu hỏi và trả lời. Một agent có thể nhận mục tiêu “kiểm tra tình trạng triển khai”, sau đó tự chọn xem log, đọc dashboard, đối chiếu commit và tổng hợp kết luận. Dù vậy, ứng dụng vẫn phải giới hạn tool, quyền và số vòng lặp.

7.2 Vòng lặp agent

1. **Observe:** nhận mục tiêu, trạng thái và kết quả tool trước đó.
2. **Decide:** mô hình chọn trả lời hoặc yêu cầu một hành động.
3. **Act:** runtime xác thực và thực thi tool.

4. **Update:** kết quả được ghi vào state/context.
5. **Stop:** kết thúc khi đạt mục tiêu, gặp giới hạn hoặc cần phê duyệt.

Agent không tự thực thi hành động chỉ vì mô hình tạo ra văn bản giống lệnh. Runtime của ứng dụng mới là thành phần kiểm tra schema, quyền và thực hiện tool.

7.3 Tool calling và function calling

Tool calling là cơ chế mô hình chọn một công cụ đã được mô tả. **Function calling** thường chỉ cách biểu diễn lời gọi dưới dạng tên hàm và tham số có cấu trúc. Trong thực tế, hai thuật ngữ thường được dùng gần nghĩa.

```
{  
  "name": "get_deployment_status",  
  "arguments": {"service": "billing-api", "environment": "staging"}  
}
```

Ứng dụng phải kiểm tra rằng service tồn tại, người dùng có quyền xem môi trường và tool chỉ đọc dữ liệu nếu đó là yêu cầu đọc. Không truyền thẳng tham số do mô hình tạo vào shell hoặc SQL.

7.4 MCP

Model Context Protocol (MCP) là chuẩn mở để ứng dụng AI kết nối với hệ thống ngoài theo mô hình client-server. Server có thể cung cấp ba loại primitive chính:

- **Resources:** dữ liệu hoặc context, như file và schema cơ sở dữ liệu.
- **Prompts:** mẫu tương tác có tham số, thường do người dùng chọn.
- **Tools:** hành động có schema mà mô hình có thể yêu cầu gọi.

MCP giải quyết lớp tích hợp, không tự giải quyết planning, evaluation, bảo mật nghiệp vụ hay chất lượng câu trả lời. Một MCP server vẫn cần xác thực, phân quyền, giới hạn dữ liệu và ghi log.

7.5 Memory và state

- **State** là dữ liệu của lần chạy hiện tại: bước đang làm, kết quả trung gian, số lần retry.
- **Short-term memory** thường là lịch sử cần thiết trong phiên.
- **Long-term memory** là thông tin lưu qua nhiều phiên, ví dụ sở thích đã được người dùng chấp thuận hoặc tri thức tổ chức.

Không phải mọi hội thoại đều cần “nhớ”. Lưu quá nhiều làm tăng rủi ro riêng tư và đưa dữ liệu cũ vào quyết định mới. Memory cần chính sách ghi, đọc, hết hạn và xóa rõ ràng.

7.6 Multi-agent

Multi-agent chia bài toán cho nhiều agent có vai trò hoặc tool khác nhau. Hai mẫu phổ biến:

- **Manager:** một agent trung tâm giữ mục tiêu và gọi các agent chuyên môn như tool.
- **Handoff:** agent hiện tại chuyển quyền điều khiển sang agent phù hợp hơn.

Chỉ nên dùng multi-agent khi ranh giới chuyên môn, quyền hoặc context thật sự tách biệt. Nếu một workflow cố định giải quyết được, multi-agent thường thêm chi phí, độ trễ và khó debug.

7.7 Khi nào không nên dùng agent?

Không cần agent khi số bước đã biết, điều kiện rõ và yêu cầu kiểm soát cao. Một workflow xác định sẽ rõ hơn, dễ test hơn và ít hành vi bất ngờ hơn. Agent phù hợp khi môi trường có nhiều đường đi hợp lệ và việc chọn bước cần suy luận từ trạng thái động.

CHƯƠNG 8

Loop Engineering và kiểm soát sai số

8.1 Loop là gì?

Loop là cơ chế lặp có điều kiện: tạo kết quả, đo kết quả, sửa và thử lại. Mục tiêu không phải để mô hình “tự suy nghĩ vô hạn”, mà để đưa tín hiệu kiểm tra cụ thể trở lại bước tạo.

Một loop tốt cần:

- điều kiện thành công đo được;
- số vòng lặp tối đa;
- lỗi được phân loại để biết có nên retry;
- giới hạn chi phí và thời gian;
- đường thoát sang fallback hoặc con người.

8.2 Reflection, critic, validator và repair

Thành phần	Nhiệm vụ	Độ tin cậy
Reflection	mô hình tự xem lại kết quả	hữu ích nhưng vẫn có cùng điểm mù
Critic	một prompt/mô hình khác tìm lỗi	tăng góc nhìn, không bảo đảm đúng
Validator	code/schema/test kiểm tra điều kiện	mạnh với tiêu chí xác định
Repair	tạo lại dựa trên lỗi cụ thể	hiệu quả khi feedback rõ

Nếu có thể viết unit test hoặc validator xác định, hãy ưu tiên nó hơn việc chỉ hỏi LLM “kết quả đã đúng chưa?”.

8.3 Ví dụ vòng lặp sửa code

1. Mô hình tạo patch.
2. Sandbox áp dụng patch vào bản sao an toàn.

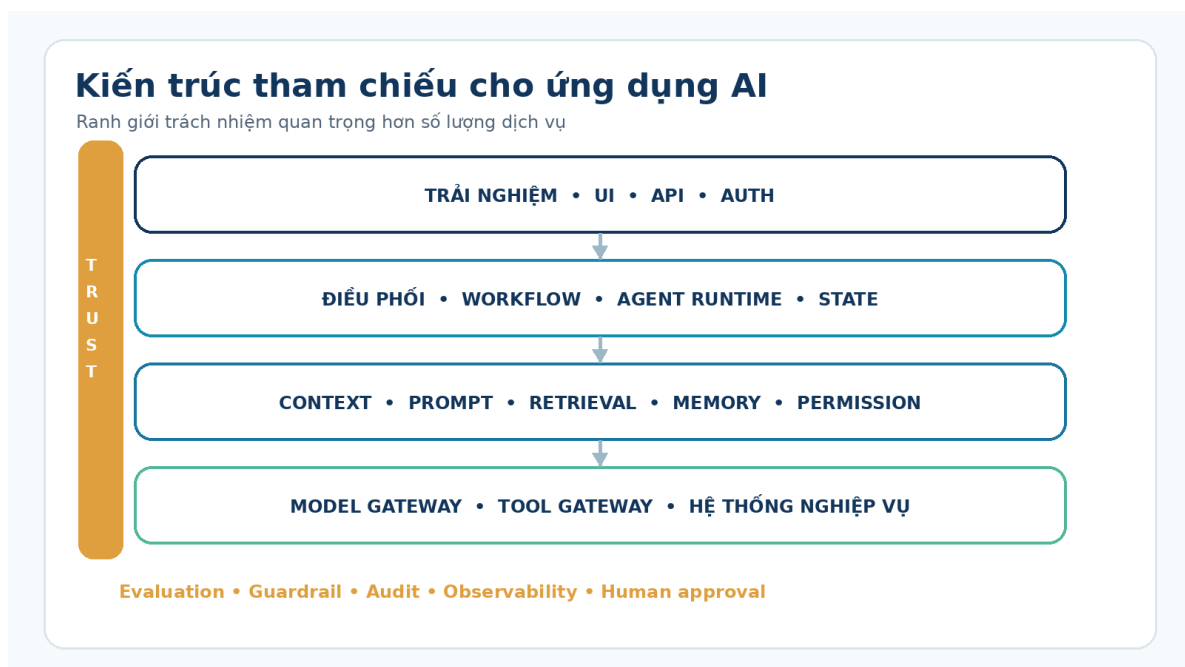
3. Linter, type checker và unit test chạy.
4. Log lỗi được rút gọn và gửi lại mô hình.
5. Mô hình sửa trong giới hạn số lần.
6. Con người review diff trước khi merge.

Loop cải thiện xác suất thành công vì dùng phản hồi từ môi trường thật. Tuy nhiên, test không đầy đủ vẫn có thể cho phép code sai nghiệp vụ vượt qua.

CHƯƠNG 9

Thiết kế hệ thống AI

9.1 Kiến trúc tham chiếu



Hình 9.1: Kiến trúc tham chiếu cho ứng dụng AI có thể kiểm soát

Một ứng dụng AI sản xuất thường có các lớp sau:

1. **Experience layer:** giao diện, API, xác thực và quản lý phiên.
2. **Orchestration layer:** workflow, agent runtime, state và routing.
3. **Context layer:** prompt, retrieval, memory, quyền và context builder.
4. **Model/tool layer:** model gateway, tool gateway, hệ thống nghiệp vụ.
5. **Trust layer:** evaluation, guardrail, audit log, observability và phê duyệt.

Sơ đồ là một khung tư duy, không bắt buộc mỗi lớp là một dịch vụ riêng. Với sản phẩm nhỏ, các lớp có thể nằm trong cùng ứng dụng nhưng vẫn nên có ranh giới mã nguồn rõ để test và thay thế.

9.2 Model gateway

Model gateway tập trung logic gọi mô hình: chọn provider, timeout, retry, rate limit, theo dõi token và chuẩn hóa lỗi. Nó giúp phần nghiệp vụ không phụ thuộc sâu vào SDK của một nhà cung cấp.

Không nên trừu tượng hóa quá sớm mọi khác biệt. Tool calling, multimodal và structured output có thể khác giữa các model. Gateway nên cung cấp interface theo nhu cầu thật của ứng dụng, không cố tạo mẫu số chung quá nghèo.

9.3 Tool gateway

Tool gateway đăng ký tool, kiểm tra tham số, áp dụng quyền, timeout và audit. Đây là ranh giới an toàn giữa output xác suất của mô hình và hành động xác định của hệ thống.

Nguyên tắc tối thiểu:

- chỉ cung cấp tool cần thiết cho nhiệm vụ;
- tách tool đọc và tool ghi;
- dùng allowlist cho tài nguyên nhạy cảm;
- yêu cầu phê duyệt với hành động không thể hoàn tác;
- không để mô hình tự tạo lệnh shell, URL hoặc SQL rồi thực thi trực tiếp;
- ghi lại ai yêu cầu, mô hình chọn gì và hệ thống đã làm gì.

9.4 Guardrail

Guardrail là tập kiểm soát nhằm ngăn hoặc phát hiện hành vi không mong muốn. Nó có thể đặt ở input, context, tool, output và hậu kiểm. Ví dụ: lọc dữ liệu cá nhân, phát hiện prompt injection, validate schema, chặn tool nguy hiểm hoặc yêu cầu con người duyệt.

Guardrail không phải một bộ lọc duy nhất và không bảo đảm an toàn tuyệt đối. Cách mạnh nhất là giảm quyền của hệ thống, phân tách trách nhiệm và thiết kế hành động có thể kiểm soát.

9.5 Prompt injection và dữ liệu không tin cậy

Prompt injection xảy ra khi nội dung không tin cậy cố hướng mô hình bỏ qua chỉ dẫn hoặc thực hiện hành động ngoài ý muốn. Nội dung đó có thể nằm trong trang web, email, PDF hoặc kết quả retrieval.

Biện pháp quan trọng:

1. xem dữ liệu truy xuất là dữ liệu, không phải instruction;
2. giới hạn tool và quyền theo người dùng;
3. xác nhận hành động ghi/xóa/gửi;
4. phân tách dữ liệu nhạy cảm khỏi context không cần thiết;
5. đánh giá bằng bộ test tấn công thực tế.

9.6 Reliability, latency và cost

Ba mục tiêu này thường đánh đổi lẫn nhau. Nhiều bước kiểm tra tăng độ tin cậy nhưng tăng độ trễ và chi phí. Model lớn có thể chính xác hơn nhưng không phải lúc nào cũng đáng dùng.

Các kỹ thuật phổ biến:

- **Routing:** dùng model nhỏ cho tác vụ đơn giản, model mạnh cho ca khó.
- **Caching:** tái sử dụng kết quả ổn định hoặc phần context tĩnh.
- **Timeout và retry:** retry có backoff cho lỗi tạm thời, không lặp vô hạn.
- **Fallback:** trả kết quả hạn chế hoặc chuyển người khi model/tool lỗi.
- **Circuit breaker:** tạm dừng luồng khi lỗi lặp hoặc chi phí vượt ngưỡng.
- **Budget:** giới hạn token, tool call, vòng lặp và thời gian cho mỗi yêu cầu.

9.7 Observability

Một trace nên cho biết input đã được rút gọn/anonymize, phiên bản prompt, model, tài liệu truy xuất, tool call, độ trễ, token, lỗi và kết quả validator. Không nên ghi bí mật hoặc toàn bộ dữ liệu nhạy cảm chỉ để debug.

Observability trả lời “đã xảy ra gì”; evaluation trả lời “kết quả có tốt không”. Hệ thống cần cả hai.

CHƯƠNG 10

Evaluation: đo chất lượng thay vì đo cảm giác

10.1 Vì sao evaluation là thành phần trung tâm?

Ứng dụng AI có đầu ra xác suất nên không thể chỉ test bằng vài ví dụ thủ công. Evaluation chuyển tiêu chuẩn sản phẩm thành tập dữ liệu và phép đo lặp lại. Nó giúp so sánh prompt, model, retrieval và workflow trước khi triển khai.



Hình 10.1: Vòng đời đánh giá và cải tiến liên tục

10.2 Xây bộ test

Một evaluation set tốt gồm:

- ca thường gặp theo phân bố thực tế;
- ca khó nhưng hợp lệ;
- ca thiếu dữ liệu, mâu thuẫn hoặc không thuộc phạm vi;
- ca tấn công, prompt injection và yêu cầu vượt quyền;

- đáp án tham chiếu hoặc tiêu chí chấm rõ;
- metadata để phân tích theo nhóm.

Không dùng toàn bộ log sản xuất chưa làm sạch làm bộ test. Cần loại dữ liệu nhạy cảm, kiểm tra quyền sử dụng và tránh để bộ test rò vào quá trình tối ưu không kiểm soát.

10.3 Các loại evaluator

Loại	Phù hợp với	Giới hạn
Rule/schema	định dạng, phạm vi số, trường bắt buộc	không hiểu sâu ngữ nghĩa
Exact/semantic match	phân loại, trích xuất, QA có đáp án	khó với nhiều đáp án hợp lệ
LLM-as-judge	chất lượng diễn đạt, bám tiêu chí phức tạp	có thiên lệch, cần hiệu chuẩn
Human review	rủi ro cao, chất lượng tinh tế	chậm và tốn chi phí
Online metric	hành vi người dùng thật	chịu nhiều yếu tố ngoài mô hình

Nên kết hợp nhiều loại. LLM-as-judge hữu ích để mở rộng chấm ngữ nghĩa, nhưng cần rubric cụ thể và đối chiếu với đánh giá của con người trên một mẫu.

10.4 Offline và online evaluation

- **Offline:** chạy trên bộ test cố định trước khi release; phù hợp regression và so sánh phiên bản.
- **Online:** đo trên lưu lượng thật qua feedback, task success, tỷ lệ sửa và A/B test.

Online metric có thể bị ảnh hưởng bởi UI, tốc độ hoặc thói quen người dùng. Cần phân biệt lỗi mô hình với lỗi toàn hệ thống.

10.5 Ngưỡng phát hành

Mỗi thay đổi prompt, model, retrieval hoặc tool nên có tiêu chí tối thiểu. Ví dụ:

- citation accuracy không giảm;
- tỷ lệ vi phạm quyền bằng 0 trên bộ security test;
- task success tăng hoặc giữ nguyên;
- p95 latency và chi phí không vượt ngân sách;
- không có regression nghiêm trọng ở nhóm ngôn ngữ cụ thể.

CHƯƠNG 11

Quy trình phát triển sản phẩm AI

11.1 Bước 1 - Xác định bài toán và ranh giới

Viết rõ người dùng, quyết định cần hỗ trợ, dữ liệu sẵn có, chi phí sai và phần nào không giao cho AI. Một mục tiêu tốt có thể đo: “giảm thời gian tìm hướng dẫn xử lý sự cố từ 15 xuống 5 phút, với citation accuracy trên 95%”.

11.2 Bước 2 - Xây baseline đơn giản

Trước khi làm agent, thử giải pháp nhỏ nhất: search từ khóa, template, model call đơn hoặc workflow hai bước. Baseline cho biết AI có thật sự tạo giá trị và cung cấp mốc so sánh.

11.3 Bước 3 - Tạo evaluation set sớm

Thu thập ca thật, định nghĩa đáp án/rubric và chạy baseline. Không chờ đến cuối mới “test AI”, vì khi đó kiến trúc có thể đã dựa trên giả định sai.

11.4 Bước 4 - Thiết kế prompt, context và tool

Chọn nguồn tri thức, quyền truy cập, schema và điểm phê duyệt. Thiết kế context builder theo ngân sách token. Tool phải có mô tả rõ, tham số hợp và lỗi dễ hiểu.

11.5 Bước 5 - Chọn workflow hoặc agent

Nếu đường đi đã biết, dùng workflow. Nếu mô hình cần chọn hành động theo trạng thái động, cân nhắc agent. Bắt đầu với quyền tối thiểu và số vòng lặp thấp.

11.6 Bước 6 - Kiểm thử nhiều lớp

- unit test cho code và tool;
- contract test cho schema;
- retrieval evaluation;

- end-to-end evaluation;
- security và permission test;
- load, latency và cost test;
- review con người cho tác vụ rủi ro.

11.7 Bước 7 - Triển khai có kiểm soát

Rollout theo nhóm nhỏ, ghi trace đã làm sạch, đặt budget và fallback. Không để phiên bản model hoặc prompt thay đổi âm thầm mà không chạy regression.

11.8 Bước 8 - Học từ lỗi thực tế

Phân loại lỗi theo lớp: dữ liệu, retrieval, model, prompt, tool, state, UI hay policy. Thêm ca lỗi đã xác minh vào evaluation set, sửa đúng lớp và đo lại. Đây là vòng lặp kỹ nghệ, không phải chỉnh prompt ngẫu hứng.

CHƯƠNG 12

Các mẫu ứng dụng AI trong thế giới thực

12.1 Trợ lý tri thức nội bộ

Bài toán: nhân viên mất thời gian tìm quy trình trong nhiều kho tài liệu.

Kiến trúc: SSO và phân quyền → hybrid retrieval → reranking → context có citation → LLM tổng hợp → feedback.

Rủi ro chính: lộ tài liệu khác phòng ban, dùng phiên bản hết hiệu lực, trích dẫn không hỗ trợ kết luận.

12.2 Intelligent Document Processing

Bài toán: chuyển PDF/ảnh hóa đơn, hợp đồng hoặc biểu mẫu thành dữ liệu có cấu trúc.

Kiến trúc: OCR/vision → phân loại → trích xuất schema → validator → đối chiếu master data → human review cho ca bất thường.

Rủi ro chính: chữ mờ, bảng phức tạp, trường hợp thiếu, số liệu hợp lệ cú pháp nhưng sai nghiệp vụ.

12.3 AI coding assistant

Bài toán: hỗ trợ đọc code, tạo patch, test và viết tài liệu.

Kiến trúc: repository context → planner → search/read tool → patch → sandbox test → review diff.

Rủi ro chính: sửa ngoài phạm vi, phụ thuộc package không an toàn, test không bao phủ, rò bí mật.

12.4 Hỗ trợ chăm sóc khách hàng

Bài toán: phân loại yêu cầu, tìm chính sách và soạn phản hồi.

Kiến trúc: classifier → customer context → policy retrieval → response draft → rule check → agent người thật khi rủi ro.

Rủi ro chính: hứa sai chính sách, tiết lộ dữ liệu, thực hiện hoàn tiền hoặc thay đổi tài khoản không có phê duyệt.

12.5 Phân tích nghiên cứu

Bài toán: tìm, nhóm và tóm tắt nhiều tài liệu.

Kiến trúc: search nguồn → lọc metadata → lưu bằng chứng → tổng hợp theo chủ đề → kiểm tra citation → con người kết luận.

Rủi ro chính: bỏ sót nghiên cứu trái chiều, nhầm tương quan với nhân quả, trích dẫn không đúng nội dung.

CHƯƠNG 13

Lựa chọn framework và công cụ

Framework giúp giảm mã lặp và cung cấp runtime, nhưng không thay thế thiết kế hệ thống. Hãy chọn sau khi đã rõ workflow, state, tool, evaluation và yêu cầu triển khai.

13.1 LangChain

Bối cảnh: cần một lớp tích hợp model, tool và middleware thống nhất cho ứng dụng LLM/agent.

Vai trò: cung cấp agent harness và các primitive để cấu hình model, tool, prompt, structured output và middleware. Các triển khai agent hiện đại của LangChain dựa trên primitive LangGraph.

Đối tượng cơ sở: create_agent, model, tool, message, middleware, structured output.

Bài toán phù hợp: xây agent bằng code với nhiều nhà cung cấp model và cần mở rộng middleware.

Lưu ý: abstraction giúp phát triển nhanh nhưng có thể che chi tiết provider; cần tracing và hiểu runtime khi debug.

13.2 LlamaIndex

Bối cảnh: dữ liệu doanh nghiệp phân tán, cần ingestion, indexing và retrieval để đưa vào LLM.

Vai trò: tập trung vào data framework cho ứng dụng RAG và agent dùng dữ liệu riêng.

Đối tượng cơ sở: document, node, index, retriever, query engine, data connector.

Bài toán phù hợp: trợ lý hỏi đáp trên tài liệu, truy vấn nhiều nguồn dữ liệu và thử nghiệm chiến lược retrieval.

Lưu ý: chất lượng vẫn phụ thuộc parsing, chunking, metadata và evaluation; framework không tự làm dữ liệu trở nên sạch.

13.3 LangGraph

Bối cảnh: agent hoặc workflow cần state, nhánh, vòng lặp, tạm dừng và tiếp tục.

Vai trò: runtime orchestration mức thấp cho workflow xác định kết hợp bước agentic.

Đối tượng cơ sở: state, node, edge, graph, checkpoint, interrupt.

Bài toán phù hợp: quy trình nghiên cứu, coding agent, human-in-the-loop và luồng chạy dài cần khôi phục.

Lưu ý: linh hoạt đi kèm trách nhiệm tự thiết kế state và điều kiện dừng.

13.4 OpenAI Agents SDK

Bối cảnh: cần runtime gọn cho agent có tool, handoff, guardrail, session và tracing.

Vai trò: quản lý vòng chạy agent, tool execution và phối hợp agent; phù hợp khi dùng trực tiếp hệ sinh thái API hỗ trợ.

Đối tượng cơ sở: Agent, Runner, tool, handoff, guardrail, session, trace.

Bài toán phù hợp: trợ lý hỗ trợ khách hàng nhiều chuyên môn, agent có tool và quy trình cần quan sát.

Lưu ý: vẫn cần permission, evaluation và thiết kế tool ở tầng ứng dụng.

13.5 MCP

Bối cảnh: ứng dụng AI và nguồn dữ liệu/công cụ cần một chuẩn kết nối dùng lại.

Vai trò: giao thức client-server cho resources, prompts và tools.

Đối tượng cơ sở: host, client, server, resource, prompt, tool và transport.

Bài toán phù hợp: cho nhiều ứng dụng AI dùng chung một lớp tích hợp file, database, SaaS hoặc công cụ nội bộ.

Lưu ý: MCP không thay thế API security, identity mapping hay policy của doanh nghiệp.

13.6 n8n

Bối cảnh: cần nối trigger và API của nhiều dịch vụ bằng workflow trực quan.

Vai trò: nền tảng automation low-code có node AI agent, model, memory, vector store và nhiều connector.

Đối tượng cơ sở: workflow, trigger, node, credential, expression, execution.

Bài toán phù hợp: xử lý email, cập nhật CRM, thông báo, tác vụ back-office có luồng rõ.

Lưu ý: quản trị credential, version workflow và kiểm thử nhánh lỗi vẫn là việc bắt buộc.

13.7 Flowise

Bối cảnh: cần thử nghiệm nhanh chatbot, RAG và agent bằng giao diện kéo thả.

Vai trò: visual builder cho luồng LLM, tích hợp model, vector store, tool và API endpoint.

Đối tượng cơ sở: chatflow/agentflow, node, credential, variable, API.

Bài toán phù hợp: prototype nội bộ và kiểm chứng kiến trúc trước khi viết dịch vụ chuyên biệt.

Lưu ý: khi lên production cần xem xét scale, secret, versioning và khả năng test tự động.

13.8 Dify

Bối cảnh: đội sản phẩm cần xây và vận hành ứng dụng LLM với giao diện quản lý tập trung.

Vai trò: nền tảng tích hợp prompt, workflow, knowledge base, tool, log và phát hành ứng dụng/API.

Đối tượng cơ sở: app, workflow, knowledge, tool, model provider, dataset/document.

Bài toán phù hợp: chatbot/assistant doanh nghiệp cần quản lý tri thức và quan sát sử dụng.

Lưu ý: cần kiểm tra mô hình phân quyền, cách dữ liệu được lưu và khả năng tùy biến theo hạ tầng.

13.9 Haystack

Bối cảnh: cần pipeline tìm kiếm và RAG mô-đun, đặc biệt khi retrieval là trọng tâm.

Vai trò: framework Python kết nối component thành pipeline, hỗ trợ document store, retriever, ranker và generator.

Đối tượng cơ sở: component, pipeline, document, document store, retriever, generator.

Bài toán phù hợp: search/RAG cấp doanh nghiệp, hybrid retrieval và pipeline cần kiểm soát từng bước.

Lưu ý: cần tự vận hành và đánh giá các component theo dữ liệu thực.

13.10 Ma trận chọn nhanh

Nhu cầu chính	Điểm khởi đầu phù hợp
Agent bằng code, cần middleware	LangChain
Data ingestion và RAG	LlamaIndex hoặc Haystack
State graph, loop, checkpoint	LangGraph
Agent runtime gọn, handoff, tracing	OpenAI Agents SDK

Nhu cầu chính	Điểm khởi đầu phù hợp
Chuẩn kết nối tool/data dùng lại	MCP
Automation nhiều SaaS, low-code	n8n
Prototype visual nhanh	Flowise
Nền tảng ứng dụng LLM tích hợp	Dify

Ma trận chỉ là điểm bắt đầu. Proof of concept nên đo trên cùng bộ dữ liệu, cùng yêu cầu bảo mật và cùng tiêu chí vận hành.

PHỤ LỤC A

Bảng tra cứu thuật ngữ

Các định nghĩa dưới đây được viết để tra cứu nhanh. Khi triển khai, nên quay lại chương liên quan để hiểu điều kiện và giới hạn.

Thuật ngữ	Nghĩa ngắn	Vai trò thực tế	Liên quan
Agent	Hệ thống để mô hình chọn hành động theo vòng lặp	Điều phối tool nhằm đạt mục tiêu	Tool, state, loop
Agent harness	Phần runtime bao quanh model	Quản lý prompt, tool, state, middleware	Agent runtime
ANN	Tìm láng giềng gần xấp xỉ	Tăng tốc vector search ở quy mô lớn	Vector store
Attention	Phép tính trọng số giữa các biểu diễn	Kết hợp thông tin trong chuỗi	Transformer
Audit log	Nhật ký hành động có thể truy vết	Điều tra ai/yêu cầu nào đã gây hành động	Governance
Autoregressive	Sinh tuần tự dựa trên phần đã có	Cơ chế phổ biến của LLM sinh văn bản	Next token
Benchmark	Bộ đo chuẩn hóa giữa các hệ thống	Sàng lọc model, không thay eval nội bộ	Evaluation
Cache	Lưu kết quả hoặc context để tái sử dụng	Giảm chi phí và độ trễ	Latency, cost
Chain	Chuỗi bước xử lý nối tiếp	Tổ chức tác vụ đơn giản	Workflow
Checkpoint	Ảnh chụp trạng thái tại một thời điểm	Khôi phục luồng chạy dài	State, durability
Chunk	Đoạn tài liệu sau khi chia	Đơn vị được lập chỉ mục và truy xuất	RAG
Chunking	Chiến lược chia tài liệu	Cân bằng độ chính xác và đủ ngữ cảnh	Retrieval

Thuật ngữ	Nghĩa ngắn	Vai trò thực tế	Liên quan
Citation accuracy	Mức độ trích dẫn thật sự hỗ trợ câu trả lời	Kiểm tra nguồn trong RAG	Grounding
Classification	Gán đầu vào vào một nhãn	Routing, kiểm duyệt, phân loại yêu cầu	ML, LLM
Completion	Phần nội dung mô hình tạo	Kết quả của một lượt sinh	Generation
Context	Thông tin mô hình nhận ở lượt hiện tại	Căn cứ để mô hình thực hiện nhiệm vụ	Prompt, RAG
Context builder	Thành phần tạo context cuối	Chọn, lọc, xếp và đóng gói dữ liệu	Context engineering
Context engineering	Thiết kế thông tin/tool đúng lúc	Tăng độ liên quan và khả năng kiểm soát	RAG, memory
Context window	Giới hạn token trong một lượt	Ràng buộc dung lượng làm việc	Token
Cosine similarity	Thuốc đo góc giữa hai vector	Ước lượng độ gần trong semantic search	Embedding
Data drift	Phân bố dữ liệu thực thay đổi	Có thể làm chất lượng giảm sau triển khai	Monitoring
Decoding	Cách chọn token từ phân phối	Điều khiển độ ổn định/đa dạng	Temperature, top-p
Deep Learning	ML dùng mạng neural nhiều tầng	Học biểu diễn phức tạp	Neural network
Diffusion model	Mô hình tạo dữ liệu qua quá trình khử nhiễu	Phổ biến trong sinh ảnh/âm thanh	Generative AI
Embedding	Vector biểu diễn đặc điểm đối tượng	Semantic search, clustering, retrieval	Vector store
Evaluation	Đo chất lượng theo tập ca và tiêu chí	So sánh, regression, release gate	Metric, dataset
Evaluator	Cơ chế chấm một kết quả	Rule, model hoặc con người	LLM-as-judge
Extraction	Lấy trường dữ liệu từ đầu vào	Chuyển văn bản/PDF thành schema	Structured output
Faithfulness	Mức câu trả lời bám nguồn đã cho	Phát hiện diễn giải không được hỗ trợ	RAG
Fallback	Phương án thay thế khi bước chính lỗi	Duy trì dịch vụ hoặc chuyển người	Reliability
Few-shot	Prompt có vài ví dụ	Dạy mẫu đầu ra tại runtime	Prompt engineering
Fine-tuning	Huấn luyện bổ sung làm đổi trọng số	Điều chỉnh hành vi hoặc miền tác vụ	Training

Thuật ngữ	Nghĩa ngắn	Vai trò thực tế	Liên quan
Foundation model	Mô hình lớn dùng làm nền cho nhiều tác vụ	Cung cấp năng lực chung	LLM, multimodal
Function calling	Mô hình tạo tên hàm và tham số có cấu trúc	Kết nối quyết định với code thực thi	Tool calling
Generative AI	AI tạo dữ liệu mới theo điều kiện	Tạo/biến đổi nội dung	LLM, diffusion
Ground truth	Nhãn hoặc đáp án tham chiếu	Cơ sở chấm evaluation	Dataset
Grounding	Neo đầu ra vào nguồn được chỉ định	Giảm suy diễn không có căn cứ	RAG, citation
Guardrail	Kiểm soát hành vi không mong muốn	Chặn, cảnh báo hoặc chuyển phê duyệt	Safety
Hallucination	Nội dung hợp lý bề mặt nhưng thiếu căn cứ/sai	Rủi ro cốt lõi của tạo sinh	Grounding
Handoff	Chuyển quyền xử lý giữa các agent	Phân tuyến theo chuyên môn	Multi-agent
HITL	Con người tham gia tại điểm kiểm soát	Duyệt hành động/rà ca rủi ro	Approval
Hybrid search	Kết hợp keyword và vector search	Tăng độ bao phủ truy xuất	BM25, embedding
Inference	Chạy mô hình để tạo dự đoán	Giai đoạn runtime	Training
Ingestion	Đưa dữ liệu nguồn vào pipeline	Chuẩn hóa trước indexing	RAG
Instruction	Chỉ dẫn hành vi/nhiệm vụ	Định hướng model	Prompt
Latency	Thời gian xử lý yêu cầu	Ảnh hưởng UX và kiến trúc	TTFT, throughput
LLM	Mô hình ngôn ngữ quy mô lớn	Lỗi hiểu/sinh ngôn ngữ của ứng dụng	Transformer
LLM-as-judge	Dùng LLM chấm output theo rubric	Mở rộng đánh giá ngữ nghĩa	Evaluation
Loop	Chu trình tạo, kiểm tra và sửa	Dùng feedback để tăng tỷ lệ đạt	Validator, retry
Machine Learning	Thuật toán học mẫu từ dữ liệu	Dự báo/phân loại/xếp hạng	AI
Memory	Dữ liệu được lưu và gọi lại qua tương tác	Duy trì thông tin cần thiết	State, context
Metadata	Dữ liệu mô tả tài liệu/bản ghi	Lọc quyền, phiên bản, phạm vi	Retrieval
Metric	Đại lượng đo chất lượng/hệ thống	Đặt ngưỡng và so sánh	Evaluation
Model gateway	Lớp chuẩn hóa việc gọi model	Routing, timeout, cost, provider	Architecture

Thuật ngữ	Nghĩa ngắn	Vai trò thực tế	Liên quan
Model drift	Hành vi model thay đổi theo phiên bản/hạ tầng	Có thể gây regression	Versioning
MoE	Kiến trúc kích hoạt một phần nhóm chuyên gia	Tăng năng lực với chi phí tính toán có chọn lọc	Model architecture
Multimodal	Xử lý nhiều loại dữ liệu	Kết hợp văn bản, ảnh, âm thanh, video	Foundation model
Multi-agent	Nhiều agent phối hợp	Tách chuyên môn hoặc quyền	Handoff, manager
Neural network	Hàm nhiều lớp có tham số học được	Nền tảng của deep learning	Training
Observability	Khả năng quan sát bên trong hệ thống	Trace lỗi, độ trễ, chi phí	Logging, tracing
One-shot	Prompt có một ví dụ	Minh họa định dạng/hành vi	Few-shot
Orchestration	Điều phối bước, state và tool	Vận hành workflow/agent	Runtime
Output parser	Chuyển output thành cấu trúc ứng dụng dùng	Phát hiện lỗi định dạng	Structured output
Parameter	Giá trị mô hình học được hoặc cấu hình runtime	Quyết định hành vi mô hình	Weight, hyperparameter
Pipeline	Chuỗi component truyền dữ liệu	Tổ chức xử lý có ranh giới	Workflow
Planner	Thành phần phân rã mục tiêu	Đề xuất bước cho tác vụ phức tạp	Agent
Prompt	Đầu vào có chủ đích cho mô hình	Chỉ định mục tiêu và điều kiện	Instruction, context
Prompt engineering	Thiết kế và kiểm thử prompt	Tăng tuân thủ không đổi weight	Few-shot, schema
Prompt injection	Dữ liệu cố chỉ phối chỉ dẫn/hành động	Rủi ro khi đọc nội dung không tin cậy	Security
RAG	Truy xuất dữ liệu rồi tăng cường context	Dùng tri thức ngoài tại runtime	Retrieval, generation
Rate limit	Giới hạn số/tốc độ yêu cầu	Bảo vệ tài nguyên và chi phí	Reliability
Recall@k	Tỷ lệ tài liệu đúng xuất hiện trong k kết quả	Đo độ bao phủ retrieval	RAG evaluation
Reranking	Chấm và sắp lại ứng viên truy xuất	Đưa đoạn liên quan lên đầu	Retrieval
Retry	Thực hiện lại sau lỗi	Xử lý lỗi tạm thời hoặc output sai	Backoff, budget
Retrieval	Tìm dữ liệu liên quan	Cung cấp bằng chứng cho context	Search, RAG

Thuật ngữ	Nghĩa ngắn	Vai trò thực tế	Liên quan
Router	Chọn model, tool hoặc nhánh	Điều phối theo dữ liệu/trạng thái	Workflow
Sampling	Lấy token từ phân phối	Tạo đầu ra cụ thể	Temperature, top-p
Semantic search	Tìm theo mức gần nghĩa	Tìm từ khóa khác nhau	Embedding
SLM	Mô hình ngôn ngữ nhỏ hơn	Giảm chi phí, chạy cục bộ/biên	Distillation, quantization
State	Dữ liệu tiến trình của lần chạy	Duy trì bước và kết quả trung gian	Workflow, agent
Structured output	Kết quả theo schema	Kết nối LLM với code ổn định hơn	JSON schema
Synthetic data	Dữ liệu được tạo nhân tạo	Bổ sung huấn luyện/test có kiểm soát	Evaluation, training
Temperature	Tham số điều chỉnh phân phối lấy mẫu	Cân bằng ổn định và đa dạng	Decoding
Token	Đơn vị rời rạc mô hình xử lý	Tính context, output và chi phí	Tokenizer
Tokenizer	Bộ chuyển chuỗi thành token ID và ngược lại	Cổng biểu diễn văn bản	Token
Tool	Chức năng ngoài model có schema	Cho hệ thống đọc hoặc hành động	Agent, function calling
Tool gateway	Lớp xác thực và thực thi tool	Phân quyền, validate, audit	Security
Top-p	Giới hạn tập token theo xác suất tích lũy	Điều khiển sampling	Temperature
Trace	Bản ghi có cấu trúc của một lần chạy	Debug từng model/tool/step	Observability
Training	Tối ưu tham số từ dữ liệu	Tạo hoặc điều chỉnh model	Inference
Transformer	Kiến trúc neural dựa trên attention	Nền tảng của nhiều LLM	Attention
TTFT	Thời gian đến token đầu tiên	Đo cảm nhận phản hồi ban đầu	Latency
Validation	Kiểm tra output theo schema/quy tắc	Ngăn dữ liệu sai đi tiếp	Guardrail
Vector database/store	Hệ lưu và tìm vector	Hạ tầng semantic retrieval	Embedding, ANN
Weight	Tham số học được của mô hình	Mã hóa các quan hệ đã học	Training
Workflow	Luồng bước/nhánh được thiết kế	Kiểm soát quá trình xử lý	Orchestration
Zero-shot	Prompt không có ví dụ mẫu	Dùng năng lực sẵn có của model	Prompt engineering

PHỤ LỤC B

Phân biệt các khái niệm thường nhầm

Khái niệm	Thay đổi cái gì?	Dùng khi nào?	Không giải quyết trực tiếp
Prompt Engineering	Cách viết chỉ dẫn và format	Nhiệm vụ chưa rõ hoặc tuân thủ kém	Tri thức nội bộ bị thiếu
Context Engineering	Thông tin/tool được nạp theo lượt	Cần đúng dữ kiện, trạng thái, quyền	Trọng số nền của model
RAG	Cách truy xuất tri thức lúc chạy	Dữ liệu lớn, cập nhật, cần citation	Hành vi/phong cách ổn định lâu dài
Fine-tuning	Trọng số model	Cần điều chỉnh mẫu hành vi/miền	Cập nhật dữ kiện thường xuyên
Workflow	Trình tự và điều kiện do hệ thống thiết kế	Đường đi đã biết, cần kiểm soát	Quyết định mở trong môi trường động
Agent	Quyền chọn hành động theo trạng thái	Nhiều đường đi hợp lệ, tool đa dạng	Bảo mật và độ đúng mặc định
Memory	Dữ liệu lưu qua bước/phiên	Cần tiếp nối thông tin đã chọn	Context tự động tốt nếu không có chính sách
Evaluation	Dữ liệu và phép đo chất lượng	So sánh, regression, release	Tự sửa hệ thống nếu không có vòng cải tiến

PHỤ LỤC C

Bản đồ trách nhiệm trong hệ sinh thái

Thành phần	Đầu vào chính	Đầu ra chính	Câu hỏi kiểm soát
Model	token + context	token/tool request	Model đủ năng lực không?
Prompt	mục tiêu + quy tắc	chỉ dẫn cho model	Nhiệm vụ đã rõ chưa?
Retrieval	query + index	tài liệu ứng viên	Có lấy đúng bằng chứng không?
Context builder	tài liệu + state	context đã chọn	Có đúng, đủ, ít nhiều không?
Workflow	input + điều kiện	state qua các bước	Đường đi và lỗi đã kiểm soát chưa?
Agent runtime	mục tiêu + tool	hành động/kết quả	Quyền và điều kiện dừng là gì?
Tool gateway	tool request	kết quả/hành động	Ai được phép làm gì?
Validator	output + rule	pass/fail/feedback	Tiêu chuẩn nào kiểm chứng được?
Evaluation	dataset + rubric	metric + lỗi	Chất lượng có cải thiện thật không?
Human reviewer	ca rủi ro + bằng chứng	quyết định chịu trách nhiệm	Khi nào cần chuyển người?

PHỤ LỤC D

Tài liệu tham khảo

D.1 Tài liệu framework và giao thức

- LangChain, LangChain overview và Retrieval: <https://docs.langchain.com/oss/python/langchain/overview>.
- LlamaIndex, Documentation: <https://docs.llamaindex.ai/>.
- LangGraph, Overview: <https://docs.langchain.com/oss/python/langgraph/overview>.
- OpenAI, Agents SDK documentation: <https://openai.github.io/openai-agents-python/>.
- Model Context Protocol, Specification: <https://modelcontextprotocol.io/specification/2025-11-25>.
- n8n, Advanced AI documentation: <https://docs.n8n.io/advanced-ai/>.
- Flowise, Documentation: <https://docs.flowiseai.com/>.
- Dify, Documentation: <https://docs.dify.ai/>.
- deepset, Haystack documentation: <https://docs.haystack.deepset.ai/docs/intro>.

D.2 Nền tảng học thuật và kỹ nghệ

- Vaswani, A. et al. (2017). Attention Is All You Need.
- Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.
- Jurafsky, D. & Martin, J. H. Speech and Language Processing.
- Kleppmann, M. Designing Data-Intensive Applications.
- Huyen, C. AI Engineering.

Kết luận

Một hệ thống AI tốt không được tạo nên chỉ bởi model mạnh. Nó được tạo bởi ranh giới trách nhiệm rõ: model xử lý phần mờ hồ; retrieval cung cấp bằng chứng; context builder chọn thông tin; workflow và runtime điều phối; tool gateway bảo vệ hành động; validator kiểm tra điều kiện; evaluation đo chất lượng; con người giữ quyền quyết định ở nơi rủi ro cao.

Khi gặp một ý tưởng AI mới, hãy vẽ luồng dữ liệu và trả lời bảy câu hỏi: đầu vào là gì, nguồn sự thật ở đâu, model chịu trách nhiệm phần nào, tool có quyền gì, lỗi được phát hiện ra sao, điều kiện dừng là gì, và metric nào chứng minh giá trị. Nếu trả lời được, bạn đã có nền tảng để tự vận dụng AI một cách có hệ thống.